

Lecture 3 -- Lecture material bullet points

- Use a **List** when order matters, and duplicate values are allowed. Use a **Set** when you only want unique values. Use a **Queue** when items need to be processed in order. Use a **Map** when you want to connect a key with a value.

Example:

```
// List
[Ali, Sara, Ali]

// Set
[Ali, Sara] // Ali is omitted.

// Queue
// Ali added --> Sara added --> John added.
[Ali, Sara, John]

//Map
[Ali --> 85, Sara --> 92]
```

- The difference between **hasNext()** and **next()** is pretty distinct. **hasNext()** checks if another item is available. It returns **true** if another item exists. It returns **false** when you reach the end. Whereas **next()** gets the next item from the collection.

```
// Example If your List is:
[Ali, Sara, John]
// The first next() gives Ali.
// The second next() gives Sara.
// The third next() gives John.
```

- **sort()** and **shuffle()** perform opposite operations. **sort()** arranges items in order whereas **shuffle()** mixes them into a random order.

```
// Example:
[8,2,5]
// Sort:
[2,5,8]
// Shuffle
[5,8,2]
```

Iterator helps you move through a collection one item at a time.

Lecture 3 -- Exercise bullet points

- **TreeSets** are always sorted. Therefore, when you try to order a **TreeSet** it will not recognize sort as a method.

```
// Create a TreeSet
TreeSet<Integer> numbers = new TreeSet<>();

numbers.add(8);
numbers.add(3);
numbers.add(1);

// THIS doesn't work since TreeSets are ALWAYS ordered.
Collections.shuffle(numbers);
```

- When you try to add a duplicate to a **TreeSet** it will output to **True**. This is really useful when combining it with conditional statements. In the case of exercise 2, it was used to check whether or not a character was a duplicate.

```
// Try to add the character to the Tree set.
// If you place a duplicate into a tree set, it will output True
// ! <-- means it checks for the True
// <-- without it, it will check for False.
// If it returns True, the duplicates int incremenents by 1
if (!characters.add(character)) {
    duplicates++;
}
```

- When creating for-loops, it's useful to know that when you're iterating over a string, you use **.length()** to determine its size. However, this is different for iterating through lists. For collections, I noticed you iterate through it's length using **.size()**. I wonder if that's the case consistently!

```
// Example for iterating through a list...
for(int i=0; i < list1.size(); i++) {
}
```

```
// Example for iterating through a string...
// This works for numbers as well (as far as I could tell).
for (int i = 0; i < text.length(); i++) {
}
```